The Transformer Encoder

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. From a black box to the real block . . . 4
4. The input: from words to vectors . . . 5
5. Inside one encoder block . . . 5
    - 5.1 Multi-head attention . . . 6
    - 5.2 Add and norm . . . 7
    - 5.3 The feed-forward network . . . 8
    - 5.4 Add and norm again, then the next block . . . 10
6. Six blocks: shapes and parameters . . . 13
7. Why each part is there . . . 14
    - 7.1 Why residual connections . . . 14
    - 7.2 Why the feed-forward network . . . 16
    - 7.3 Why 6 blocks . . . 17
8. Summary . . . 19
9. Sources . . . 19
10. Key terms . . . 20

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Multi-head attention; Positional encoding; Layer normalisation.
- **Leads to:** Transformer decoder; The transformer end to end (capstone); MLP blocks as fact storage.

1. Overview

Key point: The **encoder** (G-682) of a transformer turns a sentence into one context-aware vector per word, by passing the word vectors through 6 identical blocks of attention and a feed-forward network. Every vector keeps 512 numbers from start to end.

The transformer has two parts, an encoder and a **decoder** (G-564). The encoder first adds a positional encoding to each word's embedding, then passes the vectors through 6 identical **encoder blocks** (G-681). Each block has two **sub-layers** (G-1908): multi-head attention, then a small **feed-forward network** (G-775) that works on each word separately. Each sub-layer is wrapped in "add and norm": a **residual connection** (G-1681) adds the sub-layer's input to its output, and a layer normalisation follows.

The earlier Notes built the parts one at a time:

- self-attention replaces each word's vector by a weighted mix of the vectors of all words;
- **multi-head attention** (G-1268) runs several such mixes side by side and joins the results;
- **positional encoding** (G-1528) adds a vector for each position, so that the model knows the word order;
- **layer normalisation** (G-1054) rescales each word's vector to mean 0 and standard deviation 1.

This Note puts them together into the encoder of "Attention Is All You Need" (Vaswani et al. 2017, §3.1). The decoder is the subject of the transformer decoder.

Figure 1 is the whole architecture. Each box is one step, and the arrows show how the word vectors flow from the bottom to the top; the red arrows are residual connections, paths that skip a step and are added back. The encoder side is drawn in full. The decoder side is greyed out here.

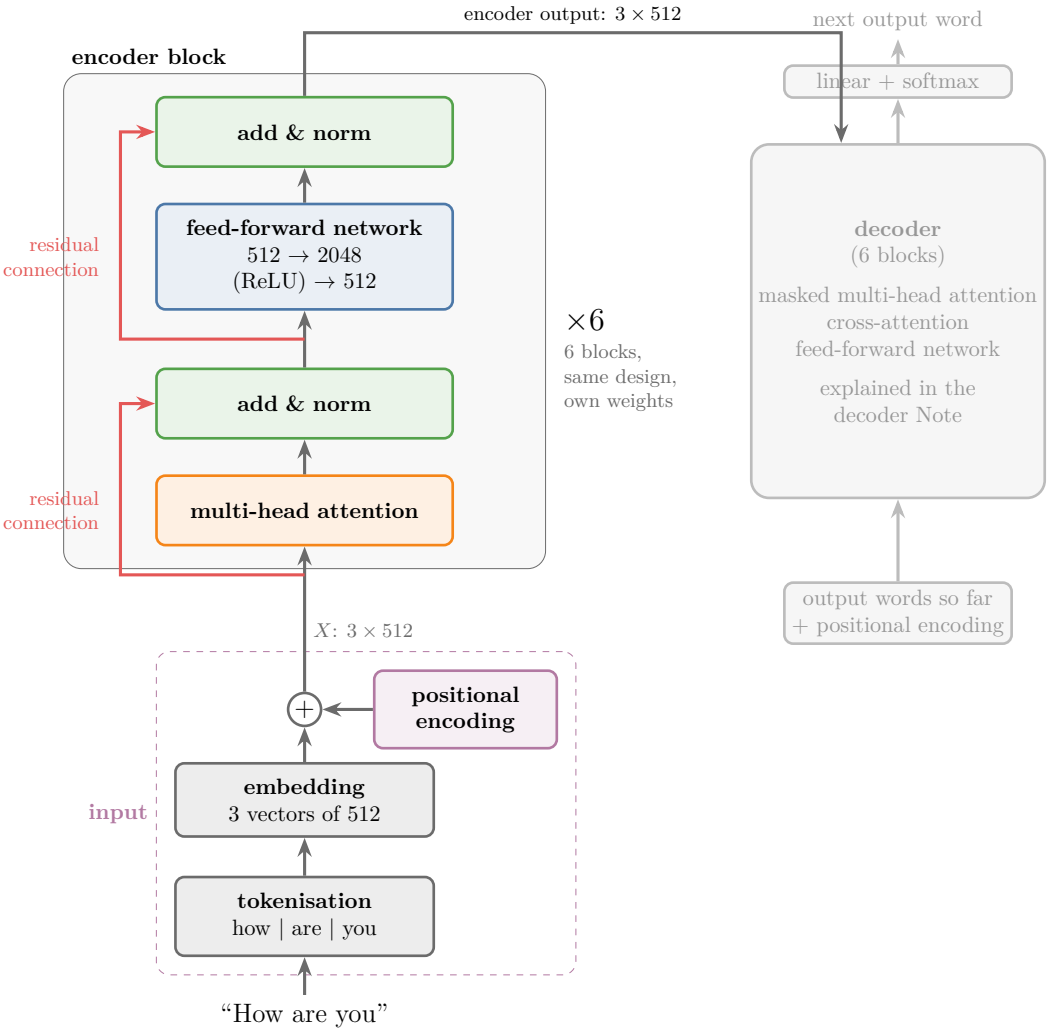


Figure 1: The transformer. Left: the input steps and one encoder block, repeated 6 times; red arrows are the residual connections. Right (grey): the decoder, which receives the encoder's output

We follow one sentence, "How are you", through the encoder. Along the way we:

- check a by-hand computation against Keras (section 5);
- count shapes and parameters (section 6);
- test why each part is there (section 7).

2. Prerequisites

- **Multi-head attention** (G-1268): 8 heads, concatenation, W_O; one 512-number output per word.
- **Positional encoding** (G-1528): a 512-number vector per position, added to the word's embedding.

- **Layer normalisation** (G-1054): each word's vector normalised by its own mean and standard deviation, then scaled by γ and shifted by β.
- Self-attention step by step: contextual embeddings from queries, keys and values.
- **Forward propagation** (G-797): a dense layer as a matrix product plus a bias, followed by an activation.
- Vanishing and exploding gradients: why deep networks are hard to train.

3. From a black box to the real block

> **Key point:** The transformer is an encoder and a **decoder** (G-564). The encoder is 6 identical blocks in a row. Each block is attention followed by a feed-forward network, with add and norm after each.

The full diagram looks complicated, so we build it up in four steps (Figure 2).

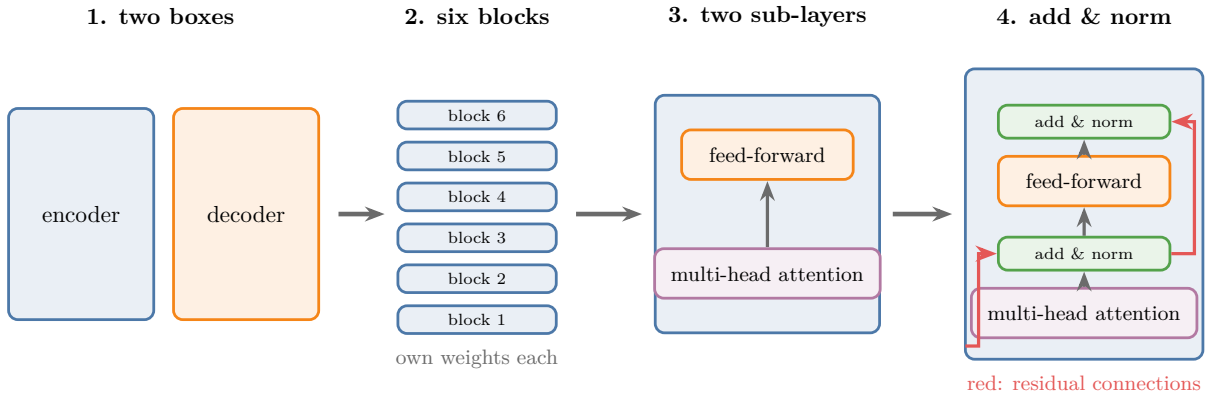


Figure 2: The encoder built up in four steps: two boxes; six blocks in the encoder; two sub-layers in each block; an add-and-norm step after each sub-layer, with the residual connections in red

1. **Two boxes.** A transformer is one box holding two boxes: the **encoder** (G-682) and the decoder. The encoder reads the input sentence; the decoder writes the output sentence.
2. **Six blocks each.** Next to both boxes the paper's diagram writes "N×". The encoder is a stack of $N = 6$ identical layers, and so is the decoder (Vaswani et al. 2017, §3.1). We call each layer an **encoder block**.
3. **Two sub-layers per block.** Each **encoder block** (G-681) has two parts: a multi-head self-attention, then a feed-forward network (Vaswani et al. 2017, §3.1).
4. **Add and norm.** Around each of the two sub-layers sits a residual connection, followed by a **layer normalisation** (G-1054): the "add & norm" boxes of Figure 1.

"Identical" means the same design. Every block has its own weights, learned separately (section 6).

The sentence enters at the bottom of the first block. The first block's output is the input of the second block, and so on. The output of the sixth block goes to the decoder.

4. The input: from words to vectors

> **Key point:** Three steps happen before the first block: split the sentence into tokens, look up a 512-number embedding for each token, and add the positional encoding of its position. The result is the input matrix X, one row per word.

The purple box of Figure 1 does three things to "How are you":

1. **Tokenisation.** The sentence is split into **tokens** (G-1981), the units the model reads. With word-level tokenisation, each token is a word: "how", "are", "you". The paper itself uses pieces of words, a method called byte-pair encoding (Vaswani et al. 2017, §5.1).
2. **Embedding.** Each token becomes a learned vector of $d_{\text{model}} = 512$ numbers (see **word embeddings** (G-2127)).
3. **Positional encoding.** The vector of each position, also 512 numbers, is added to the word's embedding, so that the model knows the word order (the **positional encoding** (G-1528)).

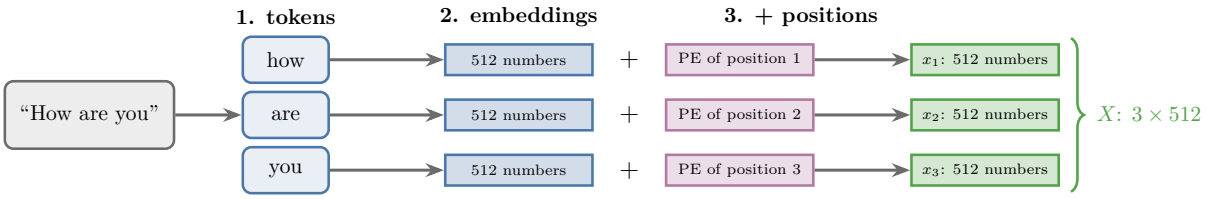


Figure 3: The three input steps for "How are you": the sentence is split into tokens, each token gets a 512-number embedding, and the positional encoding of its position is added. The three sums are the rows of X

Figure 3 follows the three steps for each word: a token is the same embedding wherever it appears, and only the added positional encoding tells the model where it stands.

The three resulting vectors x_1, x_2, x_3 are stacked as the rows of a 3×512 matrix X. In practice a batch of many sentences (a group processed together) goes in together; here the batch holds one sentence.

5. Inside one encoder block

> **Key point:** A block does four things in order: attention lets the words exchange information, an add-and-norm step adds the input back and rescales, a **feed-forward network** (G-775) transforms each word on its own, and a second add-and-norm step follows.

In symbols, the four steps are:

1. $X \to$ **multi-head attention** (G-1268) $\to Z$;
2. add X and normalise $\to Z_{\text{norm}}$;
3. feed-forward network $\to Y$;
4. add Z_{norm} and normalise $\to Y_{\text{norm}}$.

Every one of these matrices is 3×512, except the hidden layer of the feed-forward network, which is 3×2048.

Figure 4 shows the path of the three words through one block, with every shape.

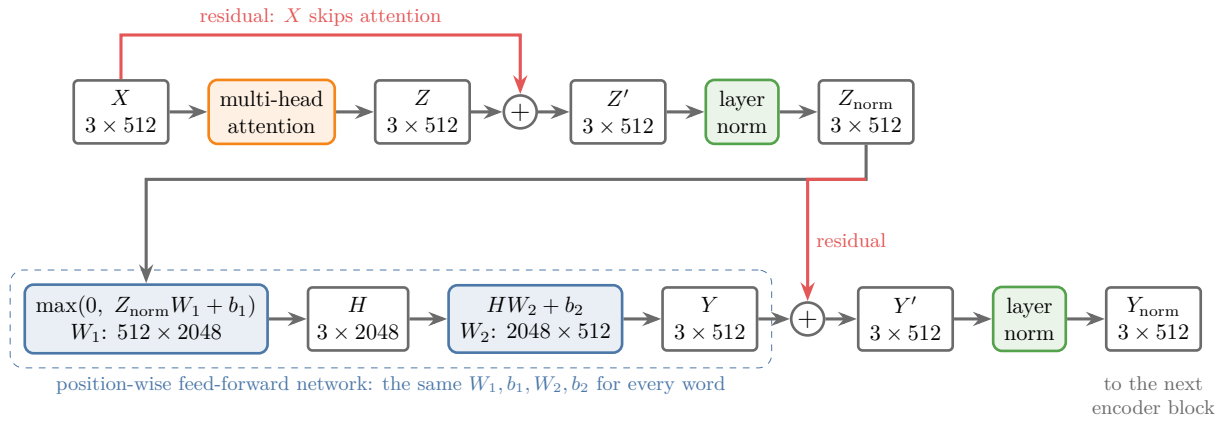


Figure 4: One encoder block for a 3-word sentence. Every matrix is 3×512 except H, the hidden layer of the feed-forward network. The red arrows are the residual connections

To show real numbers, the Notebook builds a tiny block: $d_{\text{model}} = 4$, 2 heads of 2 numbers, and a feed-forward hidden layer of 8. The embeddings of "how", "are", "you" are made-up values, and the block's weights are Keras' random starting weights. The steps and the formulas are exactly those of the full-size block. We follow the word "how", whose input row, embedding plus **positional encoding** (G-1528), is

$$x_{\text{how}} = [0.5,\ 1.0,\ -0.5,\ 0.2] + [0,\ 1,\ 0,\ 1]$$

$$x_{\text{how}} = [0.5,\ 2.0,\ -0.5,\ 1.2]$$

5.1 Multi-head attention

> **Key point:** Multi-head attention turns each word's vector into a contextual vector of the same size, using every word of the sentence.

The first **sub-layer** (G-1908) is the multi-head attention of the **multi-head attention** (G-1268), with queries, keys and values all taken from X. The embedding of a word is the

same in every sentence; attention mixes in the other words, so "bank" in "river bank" gets a different vector from "bank" in "money bank". Its output Z has one 512-number row per word, z_1, z_2, z_3, each aware of the whole sentence.

For "how" in the tiny block, the multi-head attention above (with the Notebook's weights) gives:

$$z_{\text{how}} = [1.08,\ 0.77,\ 0.39,\ -1.21]$$

5.2 Add and norm

> **Key point:** Add the sub-layer's input to its output (the **residual connection** (G-1681)), then apply layer normalisation to each word: $\text{LayerNorm}(x + \text{Sublayer}(x))$.

The paper writes the output of each **sub-layer** (G-1908) as $\text{LayerNorm}(x + \text{Sublayer}(x))$ (Vaswani et al. 2017, §3.1). Two things happen.

Add. Besides going through the attention, X also takes a second path that skips the attention (the red arrow). At the "add" box, the two meet: $Z' = X + Z$. The addition works because both are 3×512; the paper keeps every sub-layer's output at $d_{\text{model}} = 512$ "to facilitate these residual connections" (Vaswani et al. 2017, §3.1). A path that skips a sub-layer and is added back is a **residual connection** (also called a skip connection). Section 7.1 explains why it is there.

Norm. Each row of Z' is then normalised on its own: subtract the row's mean, divide by its standard deviation, multiply by γ and add β (the **layer normalisation** (G-1054)). The result is Z_{norm}.

1. **In words:** add the word's input to its attention output, then standardise the 4 (in the full model, 512) numbers of the sum.
2. **Formula:**
$$z'_i = x_i + z_i$$
$$z_{\text{norm},i} = \gamma \odot \frac{z'_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta$$
where μ_i and σ_i^2 are the mean and variance of the numbers in z'_i, ϵ is a tiny constant that keeps the division safe, γ and β are a learned scale and shift, and $\odot$ multiplies number by number.
3. **Example:** for "how",
$$z'_{\text{how}} = [0.5 + 1.08,\ 2.0 + 0.77,$$
$$-0.5 + 0.39,\ 1.2 - 1.21]$$
$$z'_{\text{how}} = [1.58,\ 2.77,\ -0.11,\ -0.01]$$
with mean $\mu = 1.06$ and standard deviation $\sigma = 1.20$. With Keras' starting values $\gamma = 1$, $\beta = 0$:
$$\frac{1.58 - 1.06}{1.20} = 0.44$$

$$\frac{2.77 - 1.06}{1.20} = 1.43$$
$$\frac{-0.11 - 1.06}{1.20} = -0.98$$
$$\frac{-0.01 - 1.06}{1.20} = -0.89$$
$$z_{\text{norm,how}} = [0.44,\ 1.43,\ -0.98,\ -0.89]$$

Attention as a change. Read the addition the other way round: the word's vector x stays, and attention adds a change z to it. The self-attention geometrically, section 7.3, draws this as an arrow e with a change Δe placed at its tip, and measures how small the change is in a trained model (Sanderson 2024, Ch 6). The **feed-forward network** (G-775) of section 5.3 adds a second change in the same way (Figure 6).

Why normalise here. The outputs of attention have no fixed range, and adding X can make them larger still. Training is more stable when the numbers stay in a small range; layer normalisation brings every word's vector back to mean 0 and standard deviation 1 before the next sub-layer (Ba et al. 2016; the **layer normalisation** (G-1054)).

5.3 The feed-forward network

> **Key point:** A two-layer network, 512 → 2048 with ReLU → 512 with no activation, applied to each word's vector separately with the same weights: $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$.

The second **sub-layer** (G-1908) is a small fully connected network (Vaswani et al. 2017, §3.3):

Layer	Nodes	Activation	Weights	Biases
input	512	—	—	—
hidden	2048	ReLU	W_1: 512 × 2048	b_1: 2048
output	512	none (linear)	W_2: 2048 × 512	b_2: 512

1. **In words:** widen each word's vector from 512 to 2048 numbers, set the negative ones to 0 (ReLU), then bring it back to 512 numbers.
2. **Formula:** for the whole matrix at once,

$$H = \max(0,\ Z_{\text{norm}}W_1 + b_1)$$

$$Y = HW_2 + b_2$$

   H has shape 3×2048 and Y has shape 3×512.
3. **Example:** in the tiny block ($4 \to 8 \to 4$), the row of "how" becomes

$$h_{\text{how}} = [0.20,\ 0,\ 0.83,\ 0.09,$$

$$0.21,\ 1.36,\ 0,\ 0.47]$$

$$y_{\text{how}} = [-0.78,\ 0.67,\ -0.14,\ -0.31]$$

Two of the eight hidden values are 0: ReLU cut them off.

The first two hidden values of "how", product by product. The row Z_{norm} of "how" is $[0.44,\ 1.43,\ -0.98,\ -0.89]$, the bias is 0, and the first two columns of W_1 (the Notebook's weights, rounded) are $[0.13,\ -0.20,\ -0.23,\ -0.23]$ and $[-0.14,\ -0.22,\ 0.45,\ -0.61]$:

$$0.44 \times 0.13 = 0.06$$

$$1.43 \times (-0.20) = -0.29$$

$$-0.98 \times (-0.23) = 0.23$$

$$-0.89 \times (-0.23) = 0.20$$

$$0.06 - 0.29 + 0.23 + 0.20 = 0.20$$

$$\max(0, 0.20) = 0.20$$

Second hidden value:

$$0.44 \times (-0.14) = -0.06$$

$$1.43 \times (-0.22) = -0.31$$

$$-0.98 \times 0.45 = -0.44$$

$$-0.89 \times (-0.61) = 0.54$$

$$-0.06 - 0.31 - 0.44 + 0.54 = -0.27$$

$$\max(0, -0.27) = 0$$

The three rows of Z_{norm} go in together, like a batch of 3 observations (records) for an ordinary network. Each row is still processed on its own. The paper calls the network **position-wise** (G-1527): it is "applied to each position separately and identically" (Vaswani et al. 2017, §3.3). The Notebook checks both halves of that sentence:

- **Separately:** feeding the three rows one at a time gives exactly the same output as feeding the whole matrix. Changing the vector of "you" changes the output of "you" (by up to 7.13) and leaves the outputs of "how" and "are" exactly as they were.
- **Identically:** the same W_1, b_1, W_2, b_2 serve every position.

Words exchange information only in the attention sub-layer: the same change to "you" alters the attention output of every word (by 1.59, 4.12 and 5.75 for "how", "are", "you").

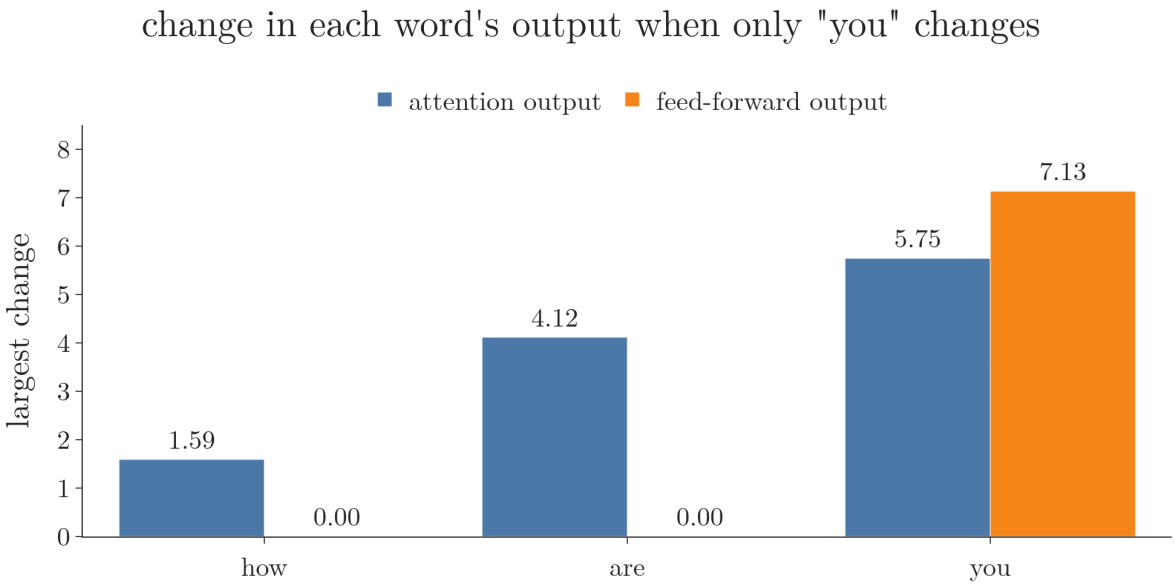


Figure 5: The tiny block of this section, with 5 added to every number of "you". Blue: how much each word's attention output changes. Orange: how much each word's feed-forward output changes

In Figure 5, the orange bars of "how" and "are" are exactly 0: the **feed-forward network** (G-775) never sees "you" when it processes them. The blue bars show attention passing the change to every word.

> **Extra:** One way to read W_1 and W_2 (Sanderson 2024, Ch 7; Geva et al. 2021):
>
> - W_1 **asks questions.** Each hidden value is the dot product of the word's vector with one column of W_1, plus a bias: how strongly the word has one particular feature.
> - **ReLU answers yes or no.** A hidden value that is negative becomes 0, so that question contributes nothing.
> - W_2 **writes the answers.** The output $y = hW_2$ is the sum of the rows of W_2, each scaled by its hidden value: every active hidden value adds its own row, a direction, to the word's vector.
>
> We write the word as a row vector times the matrix, xW_1, as the paper does. Sanderson writes the matrix times a column vector, so for him the questions are the *rows* of the first matrix and the written directions are the *columns* of the second. This reading is built up on a real model in how the feed-forward network stores facts.

5.4 Add and norm again, then the next block

> **Key point:** The feed-forward output gets its own residual connection and **layer normalisation** (G-1054). The result, still 3×512, is the input of the next block.

The second add and norm repeats section 5.2, with Z_{norm} as the input that skips the **sub-layer**

(G-1908):

$$Y' = Z_{\text{norm}} + Y$$

$$Y_{\text{norm}} = \text{LayerNorm}(Y')$$

For "how", the sum is:

$$y'_{\text{how}} = [0.44 - 0.78,\ 1.43 + 0.67,$$

$$-0.98 - 0.14,\ -0.89 - 0.31]$$

$$y'_{\text{how}} = [-0.34,\ 2.10,\ -1.12,\ -1.20]$$

After normalisation:

$$y_{\text{norm,how}} = [-0.15,\ 1.68,\ -0.73,\ -0.79]$$

Y_{norm} is the block's output. It plays the role of X for the second block, which runs the same steps with its own weights.

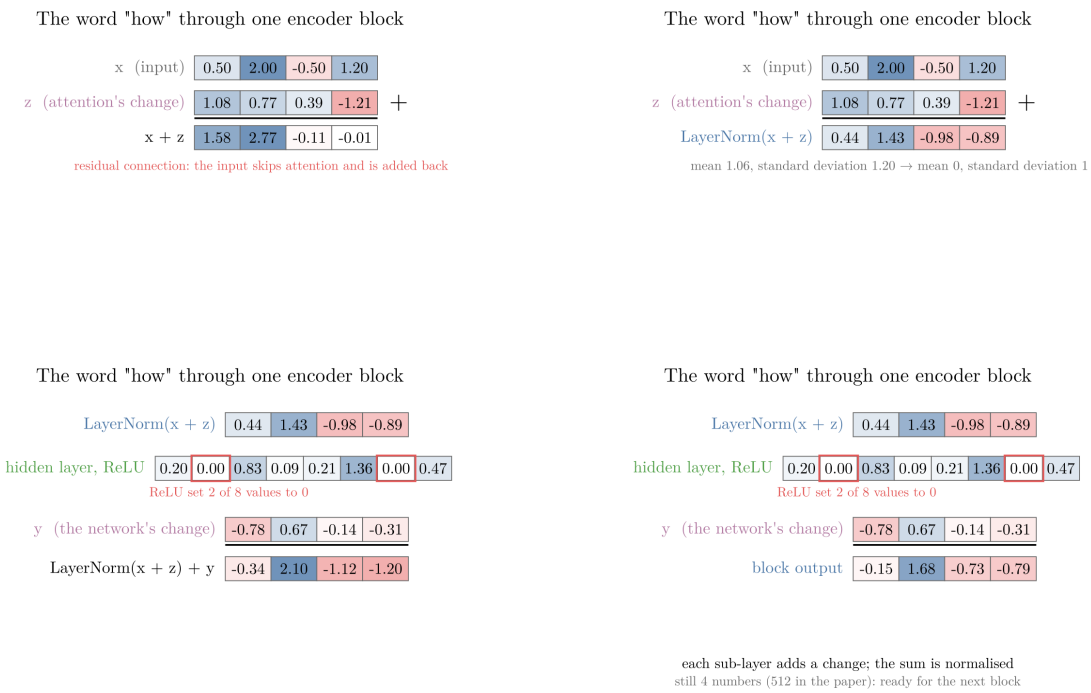


Figure 6: The word "how" through the Notebook's tiny encoder block. Attention computes a change z that is added to x; layer normalisation rescales the sum; the feed-forward network computes a second change y that is added; a second normalisation gives the block's output. Blue cells are positive, red negative

Watch in Figure 6 how each sub-layer only adds a change to the vector that comes in; the normalisation then brings the sum back to mean 0 and standard deviation 1.

The Notebook repeats the whole block by hand in NumPy, from X to Y_{norm}, and compares with the Keras block built from `MultiHeadAttention`, `LayerNormalization` and `Dense` layers. The largest difference is 5×10^{-7}: rounding error.

Python: One **encoder block** (G-681) as a Keras layer (post-norm, as in the paper).

```
class EncoderBlock(keras.layers.Layer):
    def __init__(self, d_model, heads, d_ff):
        super().__init__()
        self.mha = keras.layers.MultiHeadAttention(
            num_heads=heads, key_dim=d_model // heads)
        self.norm1 = keras.layers.LayerNormalization()
        self.ffn1 = keras.layers.Dense(d_ff, activation="relu")
        self.ffn2 = keras.layers.Dense(d_model)
        self.norm2 = keras.layers.LayerNormalization()

    def call(self, x):
        # attention, then add & norm
        z_norm = self.norm1(x + self.mha(x, x))
```

```
        # feed-forward, then add & norm
        y = self.ffn2(self.ffn1(z_norm))
        return self.norm2(z_norm + y)
```

Extra: The paper also applies dropout (the **dropout** (G-639)) with a rate of 0.1 to the output of each sub-layer, before it is added to the sub-layer's input, and to the sum of the embeddings and **positional encodings** (G-1528) (Vaswani et al. 2017, §5.4, "Residual Dropout"). Dropout only acts during training; the block above leaves it out to keep the computation exact.

6. Six blocks: shapes and parameters

Key point: The shape stays $n \times 512$ through all 6 blocks. One block has 3,152,384 parameters, two-thirds of them in the feed-forward network; the **encoder** (G-682)'s 6 blocks have 18,914,304.

Shapes. Every **sub-layer** (G-1908) maps 3×512 to 3×512, so the 6 blocks can be chained, and the encoder's output is again 3×512: one vector per input word, now informed by the whole sentence. The Notebook passes a 3×512 input through 6 Keras blocks and gets 3×512 out. For a sentence of n words, read n for 3.

Parameters. The counts follow from the layer sizes, with $d = 512$ and $d_{\text{ff}} = 2048$ (the attention formula is derived in the **multi-head attention** (G-1268), section 6.1):

Part	Formula	Parameters
Multi-head attention	$4d^2 + 4d$	1,050,624
Feed-forward network	$d \cdot d_{\text{ff}} + d_{\text{ff}} + d_{\text{ff}} \cdot d + d$	2,099,712
Two layer normalisations	$2 \times 2d$ (γ and β)	2,048
One encoder block		**3,152,384**
6 encoder blocks		**18,914,304**

Keras' `count_params()` gives exactly these numbers (Notebook). The embedding layer comes on top and depends on the vocabulary size.

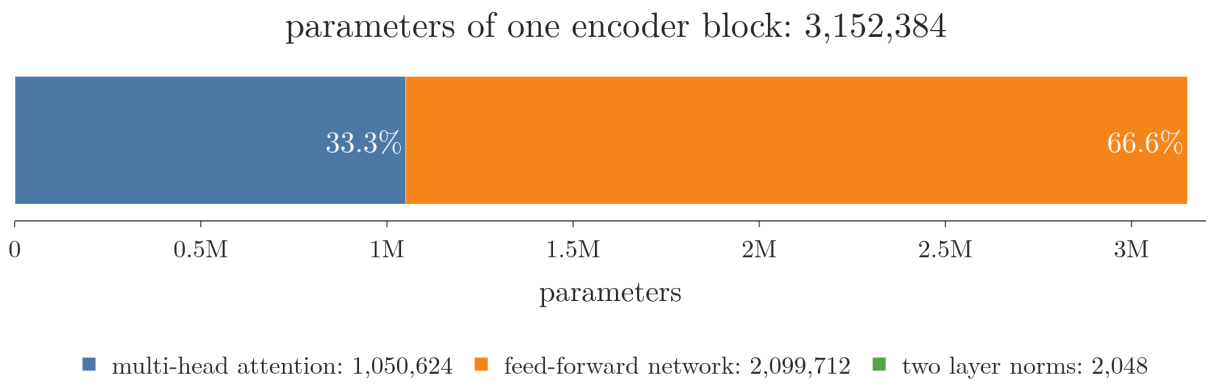


Figure 7: One encoder block's parameters split by part. The two layer normalisations hold 2,048, too few to show

The **feed-forward network** (G-775) holds 2,099,712 of a block's 3,152,384 parameters (Figure 7): 66.6 percent, two-thirds. Geva et al. (2021) open their study of these layers with the same observation: "feed-forward layers constitute two-thirds of a transformer model's parameters".

Same design, own weights. The 6 blocks are copies of one design, not of one set of numbers. Each block has its own attention matrices, its own W_1, b_1, W_2, b_2 and its own γ, β, all updated separately by backpropagation. The paper says this of the feed-forward networks: the transformations "are the same across different positions", but "they use different parameters from layer to layer" (Vaswani et al. 2017, §3.3). In the Notebook, the first entry of W_1 is 0.038 in block 1 and -0.034 in block 2.

7. Why each part is there

> **Key point: Residual connections** (G-1681) keep a direct path for the input and for the gradient through a deep stack. The feed-forward network adds a non-linear transformation of each word, and holds most of the parameters. Several blocks give the model the depth to represent language; 6 worked best among the sizes the paper tried.

The paper describes the **encoder** (G-682) but says little about why each part was chosen. The reasons below come from the papers that introduced the parts, and from experiments.

7.1 Why residual connections

> **Key point:** A **residual connection** (G-1681) lets a sub-layer's input pass straight through, so the sub-layer only has to learn a change to its input. In deep networks this makes training easier, and it keeps each word's own information from being lost. Without residual connections, a single untrained encoder block gave all 30 words of a review the same vector.

The paper uses residual connections without explaining the choice; it cites the network that introduced them, ResNet (He et al. 2016a). Two reasons are given in the literature.

Reason 1: deep stacks train more easily. He et al. (2016a) observed a **degradation problem** (G-576): when they added layers to an already deep plain network, the training error went up, not down, and the cause was not overfitting. A deeper network should never be worse on the training data, since its extra layers could simply copy their input (an identity mapping). Plain layers find copying hard. With a residual connection, a layer computes $F(x) + x$, and copying only requires $F(x) = 0$: "the solvers may simply drive the weights of the multiple nonlinear layers toward zero to approach identity mappings" (He et al. 2016a, §3.1).

1. **In words:** if a **sub-layer** (G-1908) has nothing useful to add, its weights can go to 0, and the input passes through unchanged apart from the normalisation.
2. **Formula:** with all weights and biases of the sub-layer at 0, $\text{Sublayer}(x) = 0$, so

$$\text{LayerNorm}(x + \text{Sublayer}(x)) = \text{LayerNorm}(x)$$

   Without the residual connection the same block would output $\text{LayerNorm}(0)$, which is just β for every word.
3. **Example:** for "how" in section 5.2, $x = [0.5,\ 2.0,\ -0.5,\ 1.2]$ would come out as $\text{LayerNorm}(x) = [-0.33,\ 1.30,\ -1.41,\ 0.43]$ (Notebook), still "how". Without the residual connection it would be $[0,\ 0,\ 0,\ 0]$ (with $\beta = 0$), the same for every word.

A follow-up paper showed a second effect of the identity path: the gradient of the loss with respect to an early layer contains a term that comes straight from the last layer, without passing through any weights, so it "is unlikely" to vanish, that is, shrink towards 0 so that the early layers stop learning (He et al. 2016b, §3, eq. 5).

Reason 2: each word keeps its own information. The residual connection carries the original vector forward, and each sub-layer only adds to it. The attention sub-layer can then concentrate on one job, relating the words to each other, without also having to preserve each word's embedding and position (StatQuest, "Transformer Neural Networks", 22:30). Jurafsky and Martin describe the transformer this way, as a **residual stream** (G-1683) per word: it "starts with the original input vector, and the various components read their input from the residual stream and add their output back into the stream" (SLP3 §7.2). If a sub-layer produces a poor output, the word's earlier vector is still there for the next layer to use.

Without that stream, attention alone tends to make all words alike. Dong et al. (2021) proved that a stack of attention layers without skip connections or feed-forward layers converges to an output in which every word has the same vector, and found that "skip connections play a key role in mitigating" this **rank collapse** (G-1628).

The Notebook measures it on a real IMDB review (its first 30 words, random 64-number embeddings, untrained blocks, mean of 5 random starts). After each block we compute the average cosine similarity between the words (1 means all words point the same way) and between each word and its own input vector (Figure 8).

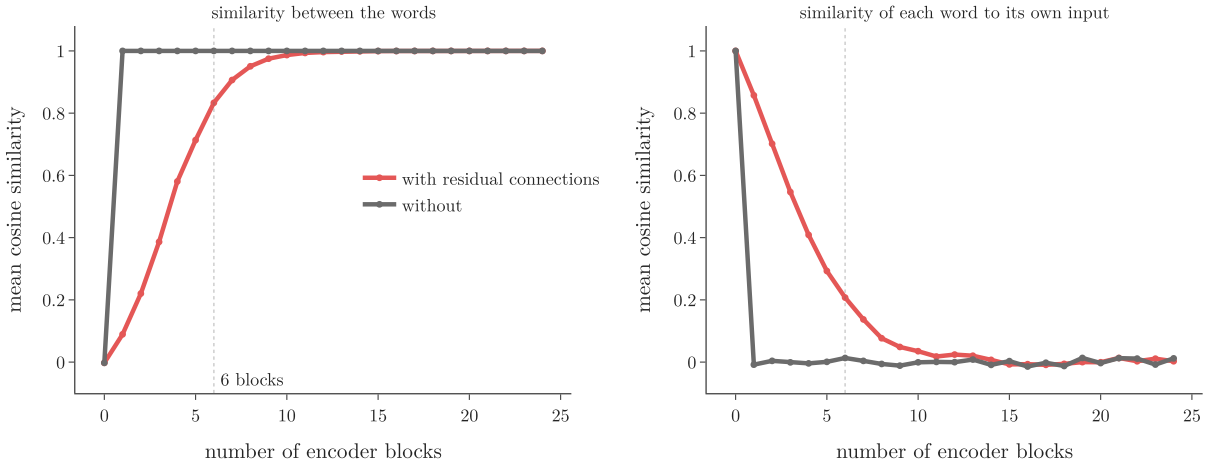


Figure 8: Untrained encoder blocks on the first 30 words of a real review, with and without residual connections, mean of 5 random starts. Left: how alike the words become. Right: how much each word still resembles its own input

Blocks	Between words, without residual	Between words, with residual	Word to own input, without	Word to own input, with
0	0.00	0.00	1.00	1.00
1	1.00	0.09	−0.01	0.86
2	1.00	0.22	0.00	0.70
6	1.00	0.83	0.01	0.21
12	1.00	1.00	0.00	0.02

Without residual connections, one block is enough: all 30 words get the same vector, and that vector has nothing left of any word's input. With residual connections the words stay distinct and close to their inputs for several blocks. In these untrained blocks the words still drift together by 12 blocks; residual connections slow the collapse rather than stop it, in line with Dong et al.'s "mitigating". Training then has a signal to work with in every word.

7.2 Why the feed-forward network

> **Key point:** Attention mixes words; the **feed-forward network** (G-775) transforms each word on its own, with a ReLU in between. Its ReLU is the block's only element-wise activation.

Attention already produces contextual vectors, so why add a second **sub-layer** (G-1908)? Two observations from the formulas:

- **Attention mixes, the feed-forward network transforms.** Each attention output is a weighted average of the value vectors, and each value vector is a linear function of the input ($v = xW_V$). The weights of that average are not linear in the input: they

come from a softmax of dot products between the words' own vectors, so attention as a whole is not a linear operation. The feed-forward network does not mix positions: it is "**position-wise** (G-1527), meaning that it operates on each token position i independently. This makes a contrast with the attention network, whose job is to mix information from different token positions" (SLP3 §7.2.1). Section 5.3 measured this split.

- **A non-linearity per word.** Once the weights are fixed, attention only averages value vectors. The ReLU inside the feed-forward network is the only activation function applied to each word's own vector in the block (the activation functions explains why stacked layers need one). It lets the block apply a non-linear transformation to each contextual vector.

What the feed-forward layers learn is an open research topic. One finding: in trained language models they behave like **key-value memories**, where the first layer's weights detect patterns in the input text and the second layer's weights push the prediction towards particular output words (Geva et al. 2021). Jurafsky and Martin add that the feed-forward parameters "seem to encode most of the factual knowledge in the transformer" (SLP3 §7.2.1).

> **Extra:** Most large language models today replace the ReLU network with a **gated** feed-forward layer, using an activation called SwiGLU, which works better than simpler functions such as ReLU (SLP3 §7.2.1).

7.3 Why 6 blocks

> **Key point:** One block is not enough to represent a language. The paper tried 2, 4, 6 and 8 blocks; 6 scored best, and other models use other numbers.

Language is complex, and a single block has limited power to represent it. Deep learning builds richer representations by stacking layers, each working on the output of the one before; the transformer stacks whole blocks.

The number 6 is an experimental choice. The paper trained versions with different numbers of blocks (Vaswani et al. 2017, Table 3, rows C, English-to-German translation, development set; BLEU measures translation quality):

Blocks N	2	4	6 (base model)	8
BLEU	23.7	25.3	25.8	25.5
Parameters (millions)	36	50	65	80

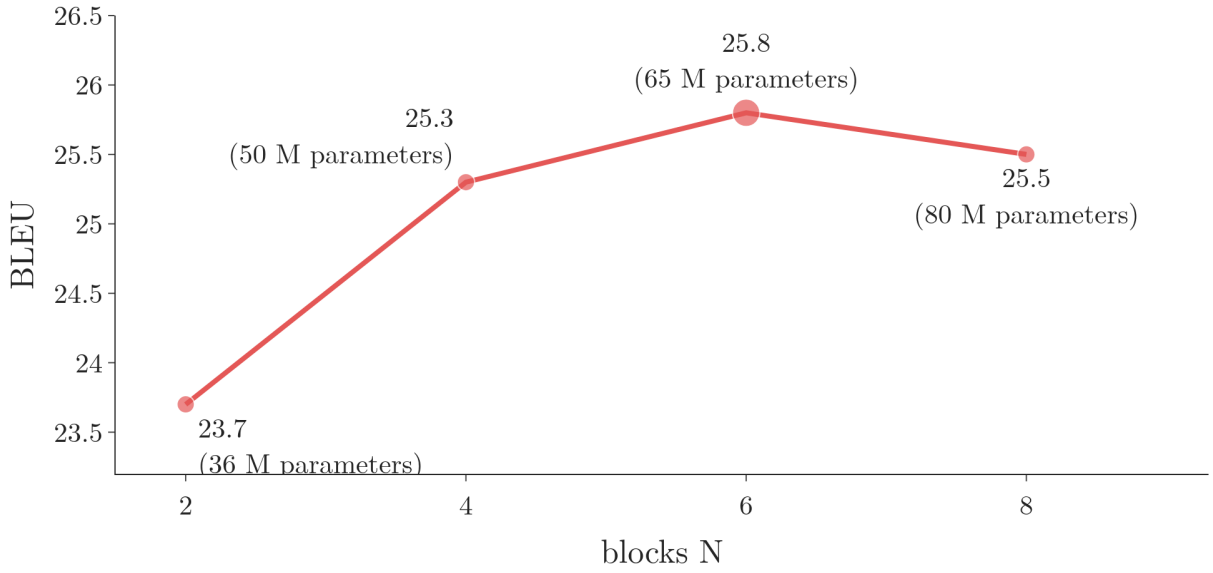


Figure 9: BLEU against the number of blocks N, with each model's size (Vaswani et al. 2017, Table 3, rows C). The base model, $N = 6$, is the largest dot

Figure 9 draws the table. Going from 2 to 6 blocks adds 2.1 BLEU; going to 8 adds parameters without a gain. The best number depends on the task and the data: BERT-base has 12 blocks and BERT-large 24 (Devlin et al. 2019, §3).

8. Summary

Step	Operation	Shape (3 words)
Input	tokens → embeddings + positional encodings	X: 3×512
Sub-layer 1	multi-head self-attention	Z: 3×512
Add & norm	LayerNorm$(X + Z)$	Z_{norm}: 3×512
Sub-layer 2	feed-forward: $\max(0, Z_{\text{norm}}W_1 + b_1)W_2 + b_2$	H: 3×2048, then Y: 3×512
Add & norm	LayerNorm$(Z_{\text{norm}} + Y)$	Y_{norm}: 3×512
Repeat	6 blocks, each with its own weights	3×512 to the decoder

- The encoder is 6 identical blocks; each block is **multi-head attention** (G-1268) and a position-wise feed-forward network, each followed by add and norm. The number 6 was chosen by experiment: it scored best of 2, 4, 6 and 8 blocks in the paper.
- Every vector keeps $d_{\text{model}} = 512$ numbers, which makes the residual additions possible and lets the 6 blocks be chained.
- One block has 3,152,384 parameters, two-thirds in the **feed-forward network** (G-775), so most of what a block learns sits there, not in attention.
- **Residual connections** (G-1681) give the input and the gradient a direct path through the stack, so each sub-layer only learns a change; without them, one untrained block gave all 30 words of a review the same vector.
- Add and norm brings each word's vector back to mean 0 and standard deviation 1, because attention outputs have no fixed range and adding the input can make them larger.
- The feed-forward network transforms each word separately, with the block's only ReLU; attention is where words exchange information, so the two **sub-layers** (G-1908) split the work between mixing words and transforming each word.
- Put together, these parts turn a sentence into one context-aware 512-number vector per word, ready for the **decoder** (G-564).

9. Sources

Built from

- CampusX, "Transformer Architecture | Part 1 Encoder Architecture | CampusX", YouTube, https://www.youtube.com/watch?v=Vs87qcdm8l0
- StatQuest with Josh Starmer, "Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=zxQyTK8quyY. 22:30 (residual connections let self-attention relate the words without also having to preserve the embedding and position).
- Sanderson, G. (3Blue1Brown), "Attention in transformers, step-by-step | Deep Learning Chapter 6", 2024, 3blue1brown.com/lessons/attention, https://www.youtube.com/watch?v=eMlx5fFN 14:45–15:44 (the attention output as a change added to the embedding).

- Sanderson, G. (3Blue1Brown), "How might LLMs store facts | Deep Learning Chapter 7", 2024, 3blue1brown.com/lessons/mlp, https://www.youtube.com/watch?v=9-Jl0dxWQs8. 7:30–13:49 (rows of the first matrix as questions, ReLU as a gate, columns of the second matrix as directions added to the vector).

Other references

- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. §3.1 (encoder: $N = 6$, two sub-layers, LayerNorm$(x + \text{Sublayer}(x))$, $d_{\text{model}} = 512$); §3.3 (position-wise feed-forward network, eq. 2, $d_{\text{ff}} = 2048$); §5.1 (byte-pair encoding); §5.4 (residual dropout, $P_{\text{drop}} = 0.1$); Table 3, rows C.
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026), ch. 7, §7.2 (transformer blocks, residual stream) and §7.2.1 (feedforward layer, SwiGLU).
- He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*. arXiv:1512.03385. §1 (degradation problem); §3.1 (identity mappings); §4.1 (plain vs residual networks).
- He, K., Zhang, X., Ren, S. and Sun, J. (2016). Identity Mappings in Deep Residual Networks. *ECCV 2016*. arXiv:1603.05027. §3, eq. 5 (the gradient's direct term).
- Dong, Y., Cordonnier, J.-B. and Loukas, A. (2021). Attention is not all you need: pure attention loses rank doubly exponentially with depth. *ICML 2021*. arXiv:2103.03404. Abstract and §1.
- Ba, J. L., Kiros, J. R. and Hinton, G. E. (2016). Layer Normalization. arXiv:1607.06450.
- Geva, M., Schuster, R., Berant, J. and Levy, O. (2021). Transformer Feed-Forward Layers Are Key-Value Memories. *EMNLP 2021*. arXiv:2012.14913. Abstract.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL 2019*. arXiv:1810.04805. §3 (BERT-base 12 layers, BERT-large 24).
- Maas, A. L. et al. (2011). Learning Word Vectors for Sentiment Analysis. *ACL 2011*. The IMDB review dataset.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Encoder block (G-681)	One layer of the transformer encoder: multi-head attention and then a feed-forward network, each followed by add and norm. It turns each word's vector into a more context-aware one; the encoder stacks 6 of them.

Term	Meaning
Sub-layer (G-1908)	One of the two parts of an encoder block: multi-head attention or the feed-forward network.
Residual connection (G-1681)	A path that skips a sub-layer and adds the sub-layer's input to its output; also called a skip connection.
Position-wise (G-1527)	Applied to each word's vector separately, with the same weights for every position.
Degradation problem (G-576)	Deeper plain networks reaching a higher training error than shallower ones.
Rank collapse (G-1628)	All word vectors of a sentence becoming the same vector after many attention layers without residual connections.
d_{ff} (G-26)	The width of the hidden layer of the transformer's feed-forward network: each word's 512-number vector is widened to $d_{\text{ff}} = 2048$ numbers, passed through ReLU, then brought back to 512.
Add and norm (G-172)	Adds a sub-layer's input back to its output (a residual connection) and then layer-normalises the sum, so the signal and gradients pass through deep stacks and the numbers stay on one scale: LayerNorm(x + Sublayer(x)).
Feed-forward network (FFN) (G-774)	A small two-layer network inside each transformer block that works on each word's vector on its own: dense layers $512 \to 2048$ with ReLU $\to 512$. Attention mixes the words; this network transforms each word and adds the block's only per-word non-linearity.
Tokenisation (tokenization) (G-1983)	Splitting a text into tokens (words or word pieces), the units a language model reads and writes, before they are turned into vectors.
Encoder (G-682)	The part of a seq2seq model that reads the input sequence and summarises it.
Decoder (G-564)	The part of a seq2seq model that writes the output sequence.
Feed-forward network (G-775)	A network in which information moves only from the first layer to the last.

Term	Meaning
Multi-head attention (G-1268)	Several self-attentions (heads) run in parallel on the same input, each with its own weights so each can capture a different kind of relation between words; their outputs are joined and mixed by one more learned matrix, W_O.
Positional encoding (G-1528)	A vector added to each word's embedding that tells the model the word's position.
Layer normalisation (G-1054)	Putting each observation's numbers on one scale by itself, without depending on the batch: subtract the mean and divide by the standard deviation across its own features, then scale and shift each feature by learned γ_j and β_j.
Forward propagation (G-797)	Passing one row of inputs through the network, layer by layer, to get the prediction.
Token (G-1981)	One unit of text the model reads or writes; here, one word.
Word embedding (G-2127)	A learned real-valued vector for each word; words used in similar ways get nearby vectors.
Dropout (G-639)	Switching off a random set of input and hidden nodes at every training step, to reduce overfitting.
Residual stream (G-1683)	The token vector that passes from block to block, to which every sub-layer adds its output.
d_{model} (G-27)	The number of values in each word's vector at the layer's input and output; 512 in the transformer.
Observation (G-1374)	One record of a dataset: one row of the data table.